

PolyLayout: Hierarchical VLM-Guided Layout Generation Beyond Rectangular Rooms

Yutong Jiang, Zahra Atashgahi, Carlos Soto Garcia Delgado, Ruben Brokkelkamp, Davide Zanutto, Efsan Sökmen, and Shahin Shahkarami

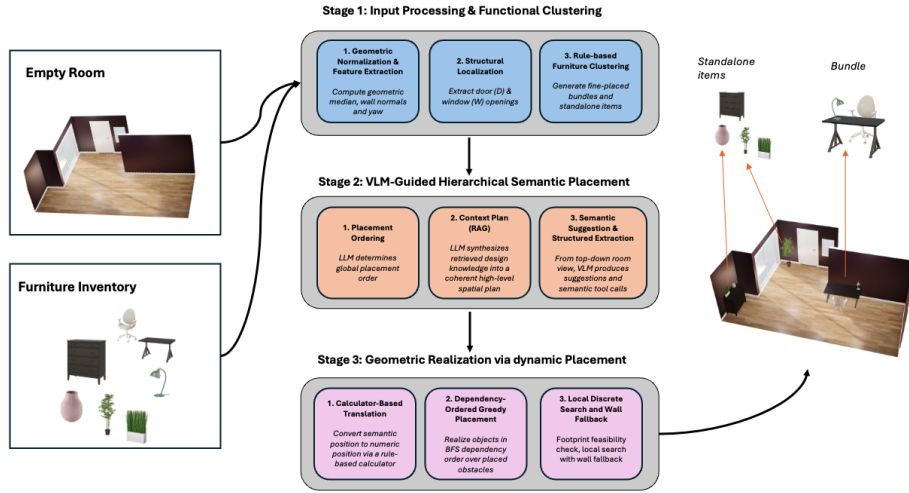
IKEA Retail (Ingka Group), Amsterdam, The Netherlands
 {yutong.jiang2,efsan.sokmen}@ingka.ikea.com

Abstract. Generating physically plausible 3D room layouts is essential for home furnishing retail, enabling customers to visualize products in their own homes and confidently make purchasing decisions. However, a gap exists between academic research and real-world application: existing solutions primarily focus on algorithmic strategies for furniture placement, largely neglecting the non-rectangular geometries and strict door/window constraints prevalent in real homes. To bridge the gap, we introduce a hybrid, hierarchical framework tailored for retail, specifically designed to support scalable spatial planning applications. Our system decouples generation into three stages: (1) functional furniture clustering and fine-grained intra-zone placement; (2) macro-routing guided by a vision-language model (VLM) to anchor both these clustered zones and any remaining standalone furniture within diverse polygonal boundaries; and (3) rule-based optimization for collision-free micro-arrangements that respect architectural constraints. We evaluate our system on production-scale catalogs and a representative set of irregular real-world topologies. Our results show that our approach attains the highest perceptual plausibility while maintaining good geometric compliance at relatively low latency, and extends to irregular boundaries that existing methods do not natively support.

Keywords: Spatial Planning · Hybrid Layout Generation · VLM

1 Introduction

Arranging furniture into functional, aesthetically coherent 3D room layouts is increasingly central to home furnishing retail, where digital planning tools support customers both online and in store. In practice, customers furnishing a space face a blank-room problem: presented with an empty floor plan, many struggle to take the first step, and abandon the planning process before engaging with the product catalog. An automatically generated initial layout lowers this barrier, offering a concrete, plausible starting point that customers can react to and refine. The core difficulty, however, is twofold: a valid layout must simultaneously respect the functional relationships between items (*e.g.*, aligning a TV with a sofa) and the hard architectural constraints of the space (*e.g.*, doors, windows, and non-rectangular boundaries).



Images © Inter IKEA Systems B.V. 2026, used with permission

Fig. 1: The POLYLayout pipeline. Stage 1 parses the room topology and groups the inventory into functional bundles with rule-based internal arrangements; Stage 2 (vision-language model, VLM) positions each bundle and standalone item over an annotated top-down rendering of the polygonal floor plan; Stage 3 resolves the proposals into precise, collision-free 3D poses in dependency order.

Traditional layout generation methods have progressed through convolutional neural network (CNN)-based placement [21], transformer architectures [22], and diffusion models [20]. However, these data-driven paradigms treat layout generation as a black-box mapping with no mechanism to enforce hard spatial constraints: small per-object errors compound into collisions or violated ergonomic relationships, and nothing guarantees that a predefined inventory is placed logically within a specific room topology.

Recent work has explored Large Language Models (LLMs) for layout generation, leveraging their spatial reasoning and natural language understanding [5, 23], and more recent efforts extend this to Vision-Language Models (VLMs) that incorporate visual context [19]. Although such models excel at semantic decisions, identifying functional zones and their relative positions, they lack geometric precision: an LLM might suggest placing a desk “against the wall,” but cannot compute exact coordinates that account for drawer clearance, chair positioning, and accessibility. Moreover, these systems confine placement to rectangular room specifications: even when doors and windows are modeled [23], the irregular polygonal boundaries ubiquitous in real homes are not treated as hard constraints. In a retail setting, however, an automated layout is unacceptable if a selected wardrobe blocks a doorway or protrudes beyond an alcove. Beyond geometric feasibility, real product catalogs carry product-specific placement rules

(*e.g.*, tall storage units must be placed against a wall to meet anchoring and tip-over safety requirements) that semantic reasoning alone cannot guarantee.

To bridge this gap, we introduce a hybrid, hierarchical framework tailored for retail (Fig. 1). Given a room topology and a predefined furniture inventory, our system decouples generation into rule-based functional clustering, VLM-driven macro-routing over the polygonal floor plan, and deterministic geometric constraint resolution.

Our contributions are as follows:

- **A hybrid hierarchical pipeline:** An end-to-end architecture that decouples layout generation into clustering, macro-routing, and micro-refinement.
- **Constraint-aware macro-routing:** A VLM-guided positioning strategy that translates irregular polygonal boundaries and fixed doors and windows into actionable spatial anchors for real-home floor plans.
- **Industry-ready micro-arrangement:** Deterministic, rule-based geometric algorithms for final constraint resolution, with the collision-aware guarantees required for retail use.

2 Related Work

Layout Generation with Deep Learning. Early data-driven approaches established foundations for indoor scene synthesis by learning spatial relationships from annotated datasets, including example-based synthesis of 3D object arrangements [6], probabilistic models [12,25] and graph-based representations [15, 26]. The introduction of deep generative models significantly advanced the field: DeepSynth [21] pioneered CNN-based sequential placement, while FastSynth [17] factorized the task into separate category, location, orientation, and dimension prediction. Transformer-based methods like ATISS [14] and SceneFormer [22] enabled autoregressive generation of object parameters, while diffusion-based approaches including DiffuScene [20] and DeBaRA [11] further improved generation fidelity.

LLM- and VLM-based Scene Generation. The emergence of Large Language Models has introduced new paradigms for controllable scene synthesis through natural language. LayoutGPT [5] was among the first to prompt LLMs to generate structured object parameters from text descriptions, and subsequent work has explored scene graph generation [1,4,7] and the direct control of procedural systems [8,10,18]. Holodeck [23] supports multi-round conversational generation that couples object selection with placement, and CityCraft [3] showed LLMs’ effectiveness in large-scale urban planning. More recently, vision-language models (VLMs) have been applied to layout generation: LayoutVLM [19] grounds placement in rendered visual context, improving spatial awareness over text-only LLMs. However, these approaches are typically demonstrated on simplified rectangular rooms, lack a mechanism to encode irregular polygonal boundaries and fixed openings as hard constraints, and fall short of the precision needed for detailed furniture arrangement.

Rule-based and Hybrid Approaches. Classical optimization-based methods [9, 13, 24] formulate layout as cost-function minimization, producing arrangements that respect functional relationships and spatial guidelines. More recently, ProcTHOR [2] and Infinigen Indoors [16] combine procedural constraints with data-driven generation to promote physical plausibility. Our work builds on this constraint-driven direction but targets the retail setting: rule-based algorithms resolve each functional bundle’s internal geometry, while a VLM positions the bundles and remaining standalone items globally over the room polygon, accounting for non-rectangular boundaries and fixed openings.

3 Methods

We formulate automated spatial planning as a decoupled, hierarchical process rather than an end-to-end black-box optimization. Section 3.1 details the core pipeline and Sec. 3.2 the fine-grained algorithms that arrange the items within each bundle.

3.1 Pipeline Architecture

Our pipeline comprises three stages (Fig. 1): input processing and rule-based functional clustering (Stage 1), VLM-guided semantic placement (Stage 2), and geometric realization via dynamic placement (Stage 3).

Stage 1: Input Processing and Functional Clustering The system accepts two inputs: room geometry in glTF format and a predefined furniture inventory of the items to be placed.

Room Feature Extraction. We distill the raw glTF scene graph into an actionable spatial representation: floor polygons, wall planes, and the vertex coordinates of structural openings. For *geometric normalization*, rather than approximating the room with an axis-aligned bounding box, we reconstruct the true floor polygon by traversing the bottom endpoints of the walls in clockwise order, which preserves non-rectangular (*e.g.* T-shaped) plans exactly. Let $\{\mathbf{v}_i\}_{i=1}^N \subset \mathbb{R}^2$ denote the ordered floor vertices on the XZ ground plane. We define the room’s *reference center* as the geometric median of these vertices (computed via Weiszfeld iterations), which, unlike the arithmetic centroid, yields a stable interior reference on elongated plans. Each wall w is assigned an inward-facing unit normal $\mathbf{n}_w$ derived from the clockwise edge orientation, and a continuous yaw $\theta_w = \text{atan2}(d_z, d_x)$ computed from its $A \rightarrow B$ direction $\mathbf{d} = (d_x, d_z)$, so that an object’s local $+X$ axis aligns with a wall at any orientation. For *structural element localization*, each door and window is associated with its parent wall via metadata, yielding per-wall opening sets $\mathcal{D}$ (doors) and $\mathcal{W}$ (windows).

Furniture Clustering. To reduce combinatorial complexity, the inventory is partitioned by a rule-based, category-driven procedure into *functional bundles*—coherent groups requiring coordinated placement (*e.g.*, a bed with nightstands)—and *standalone items*. Concretely, each catalog item carries a product-type label, and a fixed taxonomy maps 212 product types onto 43 functional roles across the six area types of Sec. 3.2. A bundle is instantiated only when its mandatory anchor role is present (*e.g.*, a bed for a sleep area), with support items attached up to per-role caps. Product types admissible in several areas (*e.g.*, rugs) are resolved by a fixed area-priority order, each instantiated bundle consuming its items before the next area type is considered; items matching no role remain standalone. Within each bundle, a fine-placement step (Sec. 3.2) arranges the constituent items relative to one another using rule-based templates (*e.g.*, positioning nightstands flush on either side of the bed), yielding a standard local configuration. Each functional bundle B_j is thereafter treated as a composite object with an aggregate bounding box (w_j, d_j, h_j) and a designated anchor.

Stage 2: VLM-Guided Semantic Placement For each functional bundle and standalone item, placement follows a semantic reasoning pipeline.

Placement Ordering. The VLM first determines a global placement order over bundles and standalone items, ensuring that large anchoring structures are positioned before items that depend on them. Wall-mounted items are deferred to the end of the order.

Context Plan via Retrieval-Augmented Generation. A high-level spatial plan is constructed via Retrieval-Augmented Generation (RAG) over curated interior-design content (furnishing guidelines, room-specific heuristics, and activity-based spatial requirements): the room context is encoded into a structured query, and the VLM synthesizes the retrieved passages into a plan specifying functional-zone allocation and inter-item relationships, which is injected into the placement prompt.

Semantic Suggestion. The VLM receives a multi-modal context: (i) a top-down rendering of the room in which wall segments carry distinct chromatic labels and structural openings are overlaid as colored markers (doors in red, windows in blue); (ii) a structured textual description containing room dimensions, per-wall door/window counts, and the wall ordering; and (iii) the dimensions of each bundle and item. To keep the output feasible, each placement selects a single `relative_position` from a reference-dependent set: wall segments admit {"center", "hang"}; the room admits its pairwise-adjacent corners, named by their two adjoining wall labels (*e.g.*, the "red-green corner"); windows admit {"under"}, doors {"next to"}, and previously placed objects {"in front", "left side", "right side", "on", "under"}, with left/right defined by the wall-orientation convention fixed in Stage 1. The full prompt template is provided in the supplementary material.

Structured Extraction. The semantic suggestions are parsed into executable placement functions:

```
<func>place_object(placement_id, [length, height, width],
                  reference_object, relative_position)</func>
```

Extraction enforces strict geometric admissibility: bundles are restricted to room corners or wall centers to prevent spatial fragmentation. An overlap-prevention rule disqualifies any wall segment containing a structural opening from “center” placement, forcing the agent to use a “next to” descriptor and preserve functional clearance.

Stage 3: Geometric Realization via Dynamic Placement The semantic suggestions are realized as precise, collision-aware 3D poses by a deterministic *dynamic placement* solver. Because each item carries an explicit structural reference, the mapping from a (**relative_position**, **reference_object**) pair to a metric pose is well-defined; the solver’s role is to locally resolve residual infeasibilities, out-of-bounds protrusion, opening obstruction, and inter-object overlap.

Calculator-Based Translation. An analytic calculator maps each structured call to a target pose $(\mathbf{t}_j^*, \theta_j)$, with anchor $\mathbf{t}_j^* = (x_j, z_j)$ and orientation $\theta_j \in [0, 2\pi)$, using the clockwise wall ordering and inward normals from Stage 1. For example: “center of [wall]” maps to the wall midpoint oriented perpendicular to the wall, and “next to [door]” to an adjacent pose preserving swing clearance.

Dependency-Ordered Greedy Placement. Objects are placed one at a time, in a dependency order given by a breadth-first traversal of the **reference_object** graph; objects already placed act as hard obstacles for the current one. A candidate pose is accepted only if it is *feasible*: the object must lie fully inside the room, keep every door and window clear, and overlap no placed object. Writing $\text{fp}(o) \subset \mathbb{R}^2$ for the XZ footprint of object o , $\mathcal{F}$ for the floor polygon, and $\mathcal{O}_{\text{placed}}$ for the set of placed objects, this feasibility test is

$$\Phi(o) = [\text{fp}(o) \subseteq \mathcal{F}] \wedge [o \text{ obstructs no opening}] \wedge [A_{\text{ovl}}(o) = 0], \quad (1)$$

where the total overlap area against placed objects is

$$A_{\text{ovl}}(o) = \sum_{o' \in \mathcal{O}_{\text{placed}}} \text{Area}(\text{fp}(o) \cap \text{fp}(o')). \quad (2)$$

Local Discrete Search and Wall Fallback. If the calculator target $\mathbf{t}^*$ already satisfies Φ , it is accepted directly. Otherwise, the solver searches a discrete set of offsets around the semantic anchor. Let $\mathcal{A}(o)$ denote the search directions of object o : the wall-parallel unit axis for wall-mounted items, plus the inward wall normal for free-standing items. The candidate poses are

$$\mathbf{t}^{(k, \hat{\mathbf{d}})} = \mathbf{t}^* + k \Delta \hat{\mathbf{d}}, \quad \hat{\mathbf{d}} \in \mathcal{A}(o), \quad k \in \{\pm 1, \dots, \pm K\}, \quad K = \left\lfloor \frac{W_{\text{wall}}}{\Delta} \right\rfloor, \quad (3)$$

with step $\Delta = 100$ mm and the deviation bounded by the reference wall width W_{wall} (a fixed 3000 mm cap is used for free-standing items lacking a wall reference). Candidates are evaluated in order of increasing $|k|$, and the first feasible pose is returned. If no zero-overlap pose exists on the original reference wall, a wall-stacked item is re-anchored to nearby walls and the search is repeated. When every candidate still overlaps, the item is either assigned the globally minimum-overlap pose

$$\mathbf{t}^* = \arg \min_{k, \mathbf{d}} A_{\text{ovl}}(o(\mathbf{t}^{(k, \mathbf{d})})), \quad (4)$$

or reported as *unplaced* rather than forced into a hard collision.

3.2 Fine-Grained Placement

The fine placement module deterministically resolves the intra-bundle geometry that semantic models cannot reliably specify: exact coordinates, clearances, and surface arrangements within each functional bundle.

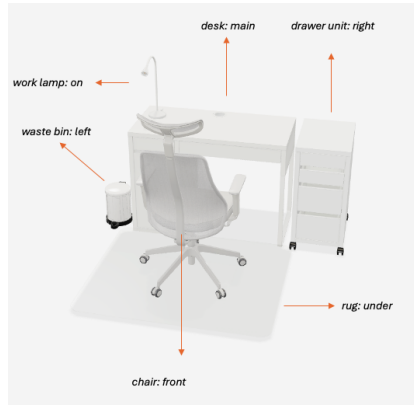
Functional Area Processing We organize intra-bundle placement into six functional area types, each governed by domain-specific algorithms and anchored by a mandatory item: work areas (desk), sleep areas (bed), dining areas (table), social areas (sofa), reading areas (seating), and storage areas (bookcase).

Rule-Based Placement Architecture

Relative Positioning. Items are positioned relative to an anchor through a relative-position template that encodes each area’s standard configuration. In a work area, for instance, the desk serves as the anchor and surrounding items follow the template (Fig. 2). These relationships are then translated into 3D coordinates (x, y, z) and rotations $(\theta_x, \theta_y, \theta_z)$, accounting for minimum clearances, accessibility, and ergonomic guidelines.

Surface Placement Geometry. For placement on surfaces (*e.g.*, desk tops), we (i) extract surface polygons from product metadata, (ii) compute the unoccupied region via polygon difference against overlapping elements, and (iii) take the axis-aligned bounding rectangle of the largest unoccupied region as the usable area, ensuring accessories are placed only on valid, stable surfaces.

Integration and Efficiency Area types are processed iteratively: after one type is placed (*e.g.*, all viable sleep areas), the remaining items are evaluated for the next, maximizing inventory utilization while preserving coherent groupings. Being deterministic and free of model inference, the arrangement completes a functional area in well under a second on CPU. The templates are, however, hand-authored: extending beyond our six area types (*e.g.*, kitchens with appliance-specific clearances) requires new domain rules, which we view as the main scalability cost of the hybrid design.



Images © Inter IKEA Systems B.V. 2026, used with permission

Fig. 2: Intra-bundle relative positioning for a work area, with the desk as the anchor.

4 Experiments

We evaluate POLYLAYOUT against two layout generators, HOLODECK [23] and LAYOUTVLM [19], on a retail benchmark of real IKEA home furnishing products in real floor plans. All three share the Gemini 2.5 Flash backbone and matched hosted configurations, isolating the placement methodology.

The three methods differ in the room geometries they natively accept. LAYOUTVLM optimizes object poses within a rectangular extent and provides no mechanism to encode openings or non-convex boundaries as hard constraints. HOLODECK does model doors and windows, but as part of its own generation: its LLM designs the floor plan and installs the doorways and windows itself [23], rather than conditioning on an externally specified room with fixed opening positions; its solver likewise assumes rectangular boundaries. We therefore structure the evaluation in two parts: a controlled three-way comparison on rectangular rooms without openings, the largest setting in which all three methods are natively applicable (Sec. 4.3), followed by a capability study of POLYLAYOUT on openings and non-rectangular plans (Sec. 4.5).

4.1 Dataset

Rooms. Only the room geometry varies across settings. We use three settings: 3 rectangular rooms without openings, used for the main comparison; 3 rectangular rooms with doors and windows; and 5 non-rectangular plans (L-, T-, U-shaped, beveled, and cut), also with doors and windows. The latter two probe generalization to real-home layouts (Sec. 4.5).

Inventories. The same 44 inventories are reused across all settings. For each room function, Gemini provides the product types; real catalog SKUs are then selected manually for those types, each with metric dimensions and 3D geometry

Table 1: Plausibility rubric used by the VLM judge, from 1 (critical failure) to 4 (polished).

Level Criterion	
1	Critical physical/boundary failure (floating, overlap, out-of-bounds)
2	A main functional group is broken
3	Coherent and functional; minor flaws tolerated (default)
4	Polished, with a clear design highlight

asset: 15 living rooms (*e.g.*, sofas, armchairs, coffee/side tables, storage, rugs, lamps *etc.*), 14 bedrooms (*e.g.*, bed frames, nightstands, wardrobes, dressers, desks, seating *etc.*), and 15 dining rooms (*e.g.*, dining tables, chairs, sideboards, lighting *etc.*), plus decor such as plants and wall art. Each inventory holds 9–33 items (mean 17.1), and each (room, inventory) pair is evaluated with two independent runs, giving $44 \times 3 \times 2 = 264$ conditions per rectangular setting and $44 \times 5 \times 2 = 440$ for the non-rectangular one.

4.2 Setup

Serving. POLYLAYOUT and HOLODECK run on CPU; LAYOUTVLM requires a GPU (1×L4), as one optimization takes minutes. Latency is end-to-end.

Metrics. We measure layouts along three axes. Completeness: **success rate** (Succ.), the fraction of conditions yielding a non-empty layout, and **placement ratio** (Place.), placed over total items. Physical validity: **in-bounds** (In-b.), the exact fraction of item footprints contained in the room polygon. Perceptual quality: **plausibility score** (Plaus.), a 4-tier score (Tab. 1) from a VLM judge (Gemini 3.1 Pro, a different model from the generation backbone; since both are Gemini models, a same-family bias cannot be fully excluded, and the human study in Sec. 4.4 serves as a model-free check). The full judge prompt is provided in the supplementary material. We also report **Latency** (lower is better), the end-to-end generation time.

Protocol. We deliberately avoid automated bounding-box overlap scores, which misclassify intentional tuck-under (chairs under tables, rugs) as collisions; gross physical failures are instead reflected in In-bounds and penalised by the plausibility judge. HOLODECK returns an empty room in 9/264 conditions, which cannot be perceptually scored; In-bounds and plausibility therefore use the $N=255$ conditions non-empty for all methods, while Success and Placement use the full 264.

4.3 Main Results

The completeness–validity trade-off. On rectangular rooms without openings (Tab. 2), success and in-bounds are near-saturated for every method (Succ.

Table 2: Main results on rectangular rooms without openings. $\uparrow/\downarrow$ mark whether higher/lower is better; Plaus. is a 1–4 Gemini 3.1 Pro judge score. In-b. and Plaus. use the $N=255$ conditions non-empty for all methods (HOLODECK returns 9 empty rooms); other metrics use all 264, as shown per column. Best in **bold**.

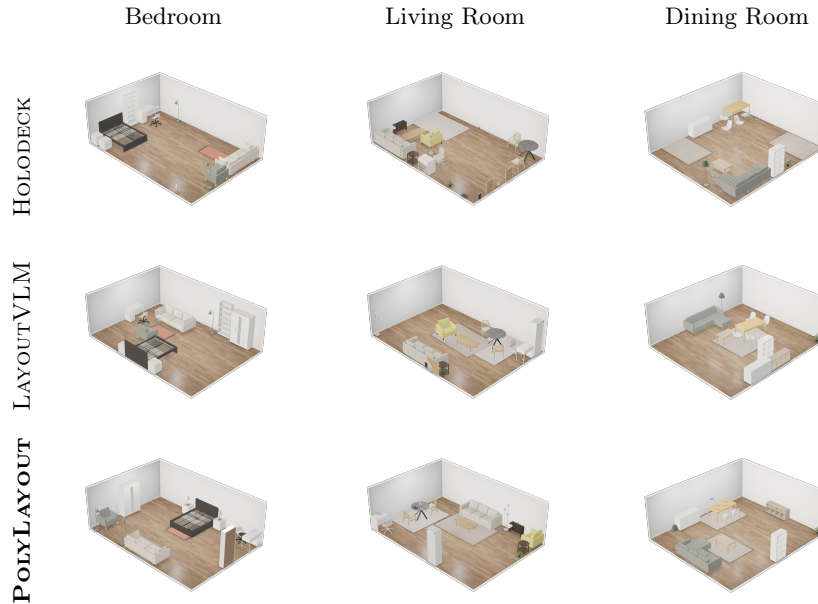
Method	Succ. (%) $\uparrow$	In-b. $\uparrow$	Plaus. $\uparrow$	Place. $\uparrow$	Lat. (s) $\downarrow$
	$N=264$	$N=255$	$N=255$	$N=264$	$N=264$
HOLODECK	96.6	1.000	2.63	0.827	62.4
LAYOUTVLM	100.0	0.996	2.49	0.989	906.8
POLYLAYOUT	100.0	1.000	2.74	0.973	70.5

$\geq 96.6\%$, In-b. ≥ 0.996), so the two discriminating axes are perceptual *plausibility* and placement *completeness*. The baselines sit at opposite ends of this trade-off. LAYOUTVLM maximises adherence, achieving the best placement ratio (0.989) by packing in nearly every object, but pays for it with the lowest plausibility (2.49): objects are placed but often collide or ignore functional relations. HOLODECK takes the opposite stance, avoiding hard overlaps through discrete constraint satisfaction; this yields cleaner but conservative scenes that drop items and, when no feasible assignment exists, return no layout at all, driving its placement ratio down to 0.827 with the highest variance ($\sigma=0.24$).

POLYLAYOUT resolves the trade-off. By resolving collisions on the discretised plan *after* VLM macro-routing, our method decouples the two axes instead of trading them off: it attains the highest plausibility (2.74) while keeping success and in-bounds perfect (100%, 1.000) and placement near-complete (0.973). Figure 3 shows representative layouts. Under paired testing the plausibility gain is significant against LAYOUTVLM (+0.25, $p < 0.001$) and marginal against HOLODECK (+0.11, $p \approx 0.05$); the blind human study (Sec. 4.4) independently confirms the ranking. Absolute performance still leaves headroom. The best plausibility (2.74) sits just below the rubric’s “coherent and functional” tier (3), consistent with our framing of generated layouts as starting points for customer refinement; inspecting low-scoring layouts, the dominant flaw traces to the product-type basis of fine placement: the taxonomy maps a fixed set of product types onto bundle roles (Sec. 3.2), so items of uncovered types are placed standalone and can drift away from their natural functional group (*e.g.*, a chair whose specific product type maps to no seating role is routed independently of the bundle), whereas broken groups are absent. On latency, POLYLAYOUT is comparable to HOLODECK (70s vs. 62s) and roughly $13\times$ faster than LAYOUTVLM (907s), though further reduction would be needed for real-time interaction.

4.4 Human Evaluation

Since plausibility score is produced by a VLM judge, we test whether these scores are grounded in what is actually rendered rather than hallucinated. Five annotators each judged the same 60 randomly sampled conditions (fixed seed),



Images © Inter IKEA Systems B.V. 2026, used with permission

Fig. 3: Qualitative comparison on three rectangular rooms. Columns share the same room and item set; rows are HOLODECK, LAYOUTVLM, and POLY LAYOUT. POLY LAYOUT keeps functional groups intact and avoids the overlaps and gaps visible in the baselines.

with the three methods’ renderings shuffled into blind, randomized slots; empty-room conditions are excluded as in the quality metrics, leaving 58 conditions (290 votes). POLY LAYOUT is preferred in 61% of votes (Tab. 3) and is the first choice of every one of the five raters, despite only fair inter-rater agreement (Fleiss’ $\kappa = 0.26$) that reflects genuinely differing tastes. Crucially, on 72% of conditions the layout humans collectively prefer is the one the judge scores highest, well above the one-third chance level, indicating the plausibility signal reflects perceived quality rather than judge hallucination. Forced-choice preference and mean plausibility measure different things (humans penalise HOLODECK’s sparsely-filled rooms more sharply than the pointwise judge), so we do not claim full-ranking agreement.

4.5 Capability Study: Openings and Non-Rectangular Plans

With inventories and algorithm fixed, we run POLY LAYOUT on progressively harder floor plans outside the scope of the controlled comparison: the three rectangular rooms without openings, three different rectangular rooms with door/window openings, and five non-rectangular (beveled/cut/L/T/U) boundaries. As discussed in the setup, neither baseline can condition on fixed openings or arbitrary polygons, and adapting their generative modules would yield

Table 3: Blind human preference (5 raters, 290 votes, empty-room conditions excluded). Best in **bold**.

Method	Votes Share (%)	
POLYLAYOUT	177	61.0
LAYOUTVLM	62	21.4
HOLODECK	38	13.1
Tie	13	4.5



Images © Inter IKEA Systems B.V. 2026, used with permission

Fig. 4: POLYLAYOUT on rectangular rooms with doors and windows. Furniture clears the openings and stays in-bounds.

a comparison reflecting our modifications rather than the original methods; we therefore read these results as a capability study of POLYLAYOUT alone: descriptive rather than comparative. Difficulty rises at each step, yet performance degrades only gently: the hard boundary constraint is never violated (in-bounds = 1.000 throughout) and average completeness stays ≥ 0.95 , while perceptual plausibility slips just $2.73 \rightarrow 2.63 \rightarrow 2.48$ (Tab. 4). The solver stays inside the polygon and fills the room in every setting, so the pipeline degrades gracefully rather than breaking as the geometry hardens. Figure 4 and Fig. 5 show example layouts with openings and on non-rectangular boundaries, respectively.

4.6 Ablation Study

To isolate the contribution of each stage of our pipeline, we ablate three core components and re-evaluate on the full benchmark under identical conditions: **A1 (w/o clustering)** removes the grouping stage that forms functional zones (*e.g.* a coordinated sofa-rug-coffee-table group); **A2 (w/o RAG plan)** removes the retrieval-augmented design plan while keeping the module that grounds suggestions into placement functions; and **A3 (w/o solver)** removes the dependency ordering and the geometric repair fallback. Results are reported in Tab. 5.

The three components degrade complementary axes rather than a single score. **Clustering (A1)** drives overall layout quality: its removal produces the largest and most significant plausibility drop and shifts the entire score distribution downward (median $3 \rightarrow 2$), consistent with the loss of coherent functional zones. **The solver (A3)** instead governs robustness: it leaves the median layout unchanged (median $3 \rightarrow 3$) yet nearly doubles the score variance ($\sigma: 0.69 \rightarrow 0.97$)



Images © Inter IKEA Systems B.V. 2026, used with permission

Fig. 5: POLYLAYOUT on the five non-rectangular boundaries (L, T, U, beveled, cut). Placements follow the irregular walls and openings without exiting the polygon.

Table 4: POLYLAYOUT across floor-plan settings (descriptive; same metrics as Tab. 2). Plaus. here averages over all N conditions rather than the matched non-empty subset of Tab. 2, hence 2.73 vs. 2.74 in the first row.

Setting	Rooms	N	In-b. $\uparrow$	Plaus. $\uparrow$	Place. $\uparrow$
Rect., no openings	3	264	1.000	2.73	0.973
Rect., +doors/windows	3	264	1.000	2.63	0.952
Non-rect., +doors/windows	5	440	1.000	2.48	0.964

and is the only variant to break the in-bounds ceiling ($1.000 \rightarrow 0.968$): removing dependency ordering and the repair fallback does not lower the typical layout but introduces a heavy tail of geometrically infeasible ones. **The RAG design plan (A2)** shows the smallest effect: its score differences are not statistically significant after correction, indicating that the retrieved guidelines polish an already-viable layout rather than determine its feasibility. Its practical value lies elsewhere: retrieval is what lets a retailer’s own design practice, such as walkway clearances and seating distances, shape the layout instead of the VLM’s generic priors, and the same interface can carry user-stated preferences toward interactive refinement.

Table 5: Ablation of the three core components. Metrics as in Tab. 2; σ is the plausibility standard deviation. Values are means with 95% confidence intervals. * marks a statistically significant change from the Full model (paired t -test, Bonferroni-corrected over the 3×3 reported comparisons, $p < 5.6 \times 10^{-3}$); n.s. otherwise. In-bounds is 1.000 for all variants except A3, where removing the solver lets a small tail of placements exit the room. All values are computed over the full 264 conditions.

Variant	In-b. $\uparrow$	Plaus. $\uparrow$	σ $\downarrow$	Place. $\uparrow$
Full model	1.000	2.73 $\pm_{.08}$	0.69	0.973 $\pm_{.008}$
A1 <i>w/o clustering</i>	1.000	2.24 $\pm_{.08}$ *	0.66	0.923 $\pm_{.012}$ *
A2 <i>w/o RAG plan</i>	1.000	2.59 $\pm_{.08}$ n.s.	0.70	0.963 $\pm_{.007}$ n.s.
A3 <i>w/o solver</i>	0.968 $\pm_{.007}$ *	2.41 $\pm_{.12}$ *	0.97	0.900 $\pm_{.014}$ *

5 Conclusion

We presented POLYLAYOUT, a hierarchical layout generator that decouples semantic arrangement (VLM-guided macro-routing grounded in design knowledge) from a deterministic geometric solver that enforces physical feasibility directly on the floor-plan polygon, including arbitrary boundaries and openings.

On a retail benchmark of real IKEA home furnishing products in real rooms with the Gemini 2.5 Flash backbone, POLYLAYOUT attains the best plausibility and perfect in-bounds placement at CPU-level latency, a ranking confirmed by both a VLM judge and a blind human study; the same pipeline extends without modification to rooms with openings and to non-rectangular L/T/U/beveled/cut plans, a regime neither baseline natively supports.

Our evaluation is limited to single rooms from the IKEA home furnishing product catalog, the cross-setting capability results are descriptive rather than controlled since each setting uses different rooms, all geometric conclusions rest on eleven rooms (six rectangular rooms, five non-rectangular rooms; condition counts scale with inventories and repeated runs rather than room variety), and no existing baseline conditions on fixed openings or polygonal boundaries, leaving the capability study without a comparator. Future work includes multi-room and whole-home layouts, evaluation across multiple VLM backbones, learning the solver’s cost terms from human preferences, and controlled comparisons as constraint-conditioned baselines emerge.

Acknowledgements

We thank our colleagues in the Home Imagination Platform team at IKEA Retail (Ingka Group) for their support, especially Thilek Silvadorai, Chris Root, as well as Bassem Samir, and Can Göçmenoglu from Geomagical Labs for their help with room builder and rendering pipeline. All furniture assets shown in figures appear courtesy of Inter IKEA Systems B.V. The IKEA logo and the IKEA wordmark are registered trademarks of Inter IKEA Systems B.V.

References

1. Çelen, A., Han, G., Schindler, K., Van Gool, L., Armeni, I., Obukhov, A., Wang, X.: I-design: Personalized LLM interior designer. In: Proc. ECCV W. vol. 15624, pp. 217–234 (2024)
2. Deitke, M., VanderBilt, E., Herrasti, A., Weihs, L., Ehsani, K., Salvador, J., Han, W., Kolve, E., Kembhavi, A., Mottaghi, R.: ProcTHOR: Large-scale embodied AI using procedural generation. In: NeurIPS. pp. 5982–5994 (2022)
3. Deng, J., Chai, W., Huang, J., Zhao, Z., Huang, Q., Gao, M., Guo, J., Hao, S., Hu, W., Hwang, J.N., Li, X., Wang, G.: Citycraft: A real crafter for 3d city generation. arXiv preprint arXiv:2406.04983 (2024)
4. Deng, W., Qi, M., Ma, H.: Global-local tree search in vlms for 3d indoor scene generation. In: CVPR. pp. 8975–8984 (2025)
5. Feng, W., Zhu, W., Fu, T.j., Jampani, V., Akula, A., He, X., Basu, S., Wang, X.E., Wang, W.Y.: Layoutgpt: Compositional visual planning and generation with large language models. In: NeurIPS. pp. 18225–18250 (2023)
6. Fisher, M., Ritchie, D., Savva, M., Funkhouser, T., Hanrahan, P.: Example-based synthesis of 3D object arrangements. ACM TOG **31**(6), 135:1–135:11 (2012)
7. Gao, G., Liu, W., Chen, A., Geiger, A., Schölkopf, B.: Graphdreamer: Compositional 3d scene synthesis from scene graphs. In: CVPR. pp. 21295–21304 (2024)
8. Hu, Z., Iscen, A., Jain, A., Kipf, T., Yue, Y., Ross, D.A., Schmid, C., Fathi, A.: Scenecraft: An LLM agent for synthesizing 3d scenes as blender code. In: ICML. pp. 19252–19282 (2024)
9. Kán, P., Kaufmann, H.: Automatic furniture arrangement using greedy cost minimization. In: Proc. IEEE Conf. Virtual Reality and 3D User Interfaces (VR). pp. 491–498 (2018)
10. Liu, J.H., Zhang, S.K., Zhang, C., Zhang, S.H.: Controllable procedural generation of landscapes. In: Proc. ACM Multimedia (MM). pp. 6394–6403 (2024)
11. Maillard, L., Sereyjol-Garros, N., Durand, T., Ovsjanikov, M.: Debara: Denoising-based 3d room arrangement generation. In: NeurIPS. vol. 37, pp. 109202–109232 (2024)
12. Merrell, P., Schkufza, E., Koltun, V.: Computer-generated residential building layouts. ACM TOG **29**(6), 181:1–181:12 (2010)
13. Merrell, P., Schkufza, E., Li, Z., Agrawala, M., Koltun, V.: Interactive furniture layout using interior design guidelines. ACM TOG **30**(4), 87 (2011)
14. Paschalidou, D., Kar, A., Shugrina, M., Kreis, K., Geiger, A., Fidler, S.: Atiss: Autoregressive transformers for indoor scene synthesis **34**, 12013–12026 (2021)
15. Qi, S., Zhu, Y., Huang, S., Jiang, C., Zhu, S.C.: Human-centric indoor scene synthesis using stochastic grammar. In: CVPR. pp. 5899–5908 (2018)
16. Raistrick, A., Mei, L., Kayan, K., Yan, D., Zuo, Y., Han, B., Wen, H., Parakh, M., Alexandropoulos, S., Lipson, L., Ma, Z., Deng, J.: Infinigen indoors: Photorealistic indoor scenes using procedural generation. In: CVPR. pp. 21783–21794 (2024)
17. Ritchie, D., Wang, K., Lin, Y.a.: Fast and flexible indoor scene synthesis via deep convolutional generative models. In: CVPR. pp. 6182–6190 (2019)
18. Sun, C., Han, J., Deng, W., Wang, X., Qin, Z., Gould, S.: 3D-GPT: Procedural 3D modeling with large language models. In: 2025 International Conference on 3D Vision (3DV). pp. 1253–1263 (2025)
19. Sun, F.Y., Liu, W., Gu, S., Lim, D., Bhat, G., Tombari, F., Li, M., Haber, N., Wu, J.: LayoutVLM: Differentiable optimization of 3D layout via vision-language models. In: CVPR. pp. 29469–29478 (2025)

20. Tang, J., Nie, Y., Markhasin, L., Dai, A., Thies, J., Nießner, M.: Diffuscene: Denoising diffusion models for generative indoor scene synthesis. In: CVPR. pp. 20507–20518 (2024)
21. Wang, K., Savva, M., Chang, A.X., Ritchie, D.: Deep convolutional priors for indoor scene synthesis. ACM TOG **37**(4), 70:1–70:14 (2018)
22. Wang, X., Yeshwanth, C., Nießner, M.: Sceneformer: Indoor scene generation with transformers. In: 2021 International Conference on 3D Vision (3DV). pp. 106–115 (2021)
23. Yang, Y., Sun, F.Y., Weihs, L., Vanderbilt, E., Herrasti, A., Han, W., Wu, J., Haber, N., Krishna, R., Liu, L., Callison-Burch, C., Yatskar, M., Kembhavi, A., Clark, C.: Holodeck: Language guided generation of 3d embodied AI environments. In: CVPR. pp. 16277–16287 (2024)
24. Yu, L.F., Yeung, S.K., Tang, C.K., Terzopoulos, D., Chan, T.F., Osher, S.J.: Make it home: automatic optimization of furniture arrangement. ACM TOG **30**(4), 86 (2011)
25. Yu, L.F., Yeung, S.K., Terzopoulos, D.: The clutterpalette: An interactive tool for detailing indoor scenes. IEEE TVCG **22**(2), 1138–1148 (2015)
26. Zhang, S.H., Zhang, S.K., Xie, W.Y., Luo, C.Y., Yang, Y.L., Fu, H.: Fast 3d indoor scene synthesis by learning spatial relation priors of objects. IEEE TVCG **28**(9), 3082–3092 (2022)

Supplementary Material

This supplementary material provides the full prompt templates of the POLY-LAYOUT pipeline (Appendix A) and of the VLM plausibility judge used in our evaluation (Appendix B). Curly-braced fields (*e.g.*, {room_type}) are template variables filled at runtime with the room context described in the main paper.

A Stage-2 Prompt Templates

Stage 2 of the pipeline (Sec. 3.1 of the main paper) issues four VLM calls in sequence: placement ordering, RAG-based context planning, semantic suggestion, and structured extraction. A preprocessing call classifies wall-mounted items. The exact templates follow.

A.1 Placement Ordering

Determines the global placement order over functional bundles and standalone items (“Placement Ordering” in the main paper). Bundles are referred to as *bubbles* in the implementation.

```
<instruction>
You are an expert interior designer. Your task is to determine the placement priority for a
    ↳ list of furniture items and functional bubbles to be placed in a {room_type}.

The room may already contain placed items (bubbles and single furniture). These placed
    ↳ elements are provided only for context. You must prioritize ONLY the elements that
    ↳ still need to be placed.

The placement system works iteratively: it places the most important items first. If an item
    ↳ cannot be placed, it is skipped. Therefore, the order you define is critical to
    ↳ ensure the most essential elements are included in the final layout.

Your goal is to order all to-place elements from most important to least important, ensuring
    ↳ functional balance for the room.
</instruction>

<context>
-----
ROOM INFORMATION
-----
- Room Type: {room_type}
- Walls: {walls_dimensions_query}
- Window(s): {window_information}
- Door(s): {door_information}

-----
PLACED ELEMENTS (CONTEXT ONLY)
-----
- Placed Functional Bubbles: {placed_formatted_bubble_list}
- Placed Single Furniture Items: {placed_formatted_single_furniture_list}

-----
ELEMENTS TO ORDER (TO PLACE)
-----
- Functional Bubbles: {to_place_formatted_bubble_list}
- Single Furniture Items: {to_place_single_furniture_list}
</context>
```

```

<output_format>
Return a single, flat Python list containing ONLY the names of all to-place bubbles and
    ↪ to-place single furniture items in their prioritized order. The list should contain
    ↪ nothing but the string names of the to-place elements.
</output_format>

<example>
# Input (Empty Room Case):
# placed_bubble_list: []
# placed_furniture_list: []
# to_place_bubble_list: ["sleeping_area_1"]
# to_place_furniture_list: ["rug_1", "nightstand_1", "wardrobe_1", "bed_2"]
# room_type: "Bedroom"

# Output:
["sleeping_area_1", "wardrobe_1", "nightstand_1", "rug_1", "bed_2"]
</example>

<example>
# Input (Partially Furnished Case):
# placed_bubble_list: ["resting_area_1"]
# placed_furniture_list: ["tv_unit_1"]
# to_place_bubble_list: ["eating_area_1"]
# to_place_furniture_list: ["table_with_four_chairs_1", "three_place_sofa_1",
    ↪ "coffee_table_1", "rug_1", "floor_lamp_1", "drawer_1", "beige_sofa_1",
    ↪ "beige_sofa_2", "frame_1", "frame_2"]
# room_type: "Small Living-Dining Room"

# Output:
["eating_area_1",
 "table_with_four_chairs_1",
 "three_place_sofa_1",
 "rug_1",
 "coffee_table_1",
 "floor_lamp_1",
 "drawer_1",
 "beige_sofa_1",
 "beige_sofa_2",
 "frame_1",
 "frame_2"]
</example>

<instruction>
Now, generate the prioritized Python list for ONLY the to-place elements provided.
Use the placed elements as context to maintain functional balance but do not include them in
    ↪ the output list.
</instruction>

```

A.2 Context Plan via Retrieval-Augmented Generation

Synthesizes the retrieved interior-design guidelines ({context_guidelines}) into a high-level spatial plan (“Context Plan via Retrieval-Augmented Generation” in the main paper).

```

<instruction>
You are a module in charge of generating a high level plan about how to place furniture in a
    ↪ room.

REFERENCE OBJECT DEFINITIONS
Walls: [{wall_list}]
Windows: [{window_list}]
Doors: [{door_list}]
Room type: {room_type}

```

```

The room may already contain placed elements (bubbles and single furniture). These are
    ↳ context and should influence how you plan the placement of the remaining to-place
    ↳ elements.

Elements to consider:
- Placed (Context Only)
  - Bubbles: {placed_formatted_bubble_list}
  - Single Furniture: {placed_formatted_single_furniture_list}
- To Place (Plan for these)
  - Bubbles: {to_place_formatted_bubble_list}
  - Single Furniture: {to_place_single_furniture_list}

Elements have an order of importance defined in the following list: {ordering_list}

Wall information: {walls_dimensions_query}
Window information: {window_information}
Door information: {door_information}

Your goal is to create a high level plan that outlines how to place the TO-PLACE furniture
    ↳ into the room given the context of already placed elements.
Do not provide specific placements, numeric values or function calls. Focus on the overall
    ↳ strategy and considerations for arranging the to-place furniture effectively into
    ↳ the provided room.
You need to provide a plan for all TO-PLACE furniture objects (bubbles and single
    ↳ furniture). Refer to each bubble as a whole, not individual furniture within the
    ↳ bubble.

CRITICAL -- DOOR & WINDOW CLEARANCE:
- The image shows doors (red) and windows (blue) with their positions clearly marked. The
    ↳ room dimensions (wall lengths and distances) are also annotated.
- Use spatial awareness: consider the size of furniture relative to the available wall space
    ↳ between openings.
- NEVER place furniture that blocks or overlaps with a door's swing area or passage zone.
- NEVER place furniture directly in front of a window in a way that blocks access or
    ↳ overlaps with the window opening area.
- Treat the full width of each door and window as a no-furniture zone.
- When placing furniture "next to" a window or door, ensure it is beside the opening, not
    ↳ covering it.
- Be mindful of how much free wall space remains after accounting for doors and windows --
    ↳ do not overcrowd narrow wall segments.

For context, consider the following guidelines for placing furniture:
{context_guidelines}
{layout_style_directive}

IMPORTANT OUTPUT FORMAT:
- Keep your response CONCISE: maximum 300 words.
- Use brief bullet points, not lengthy paragraphs.
- State WHICH bubble goes on WHICH wall/corner, and WHY in one sentence.
- Do NOT repeat the room geometry or furniture dimensions back.
- Do NOT write section headers, markdown formatting, or numbered sub-sections.

```

A.3 Semantic Suggestion

The multi-modal placement prompt (“Semantic Suggestion” in the main paper). It accompanies the annotated top-down rendering of the room and produces one free-text placement rationale per element.

```

<instruction>

You are an interior designer tasked with placing furniture items in a room.

```

```

-----
ROOM CONTEXT
-----
1. Room Context
Room Type: {room_type}
Nr. of Walls: {nr_walls}
Walls: {walls_dimensions_query}
Window(s): {window_information}
Door(s): {door_information}

-----
CURRENT LAYOUT
-----
2. Current Layout
Top-Down View:
You are provided with a top-down image of the room showing its current state.

Furniture can come in two different forms:
-Single furniture item: A single piece of furniture
-Bubble: A group of single furniture items with a fixed position relative positions

Furniture items to place: {furniture_list}
Functional Bubbles to Place: {bubble_list}

- Relative positions within each bubble: {relative_pos_store}

Elements have an order of importance defined in the following list: {ordering_list}

Your task is to suggest placements for **each bubble and each single furniture item in the
    ↪ single furniture items list** based on the room layout and existing furniture.
<context>
Another expert has provided placement suggestions that you can use as context: {context_plan}
</context>

</instruction>

<constraint>
Follow these design and placement rules:

1. Base all placements on room dimensions, existing furniture, and good design principles.
2. If multiple elements are provided, place them in a logical sequence. Place large elements
    ↪ first then smaller elements. If you want to position one object relative to
    ↪ another, make sure the reference object is placed earlier in the order.
3. For each element, provide a clear placement explanation wrapped in '<reas>...</reas>'
    ↪ tags **only**.
4. You can reference:
    - Existing or previously placed elements
    -- Room itself: "room"
    - Wall: [{wall_list}]
    - Window: [{window_list}]
    - Door: [{door_list}]
5. Valid positions relative to the **reference object**:
    - If reference object is existing or previously placed furniture: the reference position
    ↪ should be chosen from ["in front", "left side", "right side", "on", "under"].
    - If reference object is wall: the reference position should be chosen from ["center",
    ↪ "hang"]. NOTE: "hang" is ONLY for wall-mounted items. Floor furniture MUST use
    ↪ "center".
    - If reference object is room: the reference position should be chosen from
    ↪ [{corner_list}]
    - If reference object is window: reference position should be chosen from ["next to"]
    - If reference object is door: reference position should be chosen from ["next to"]
5a. Disambiguation of "left side" and "right side":
    - Imagine traversing the room perimeter in a clockwise direction.
    - When you reach (conceptually align with) a referenced furniture item, the side of that
    ↪ item you would encounter FIRST in that clockwise traversal is its "right side".
    - The opposite lateral side (encountered second) is its "left side".
    - Always apply this clockwise rule; do not use a viewer-centric or camera perspective.

```

Wall center vs corner leverage:

- Choosing a wall "center" fixes the element at the midpoint, clarifying which wall segment
→ is occupied.
- Choosing a room corner places the element squarely in that corner, reserving adjacent wall
→ centers for other functions.

Use this deliberately to optimize spatial distribution and avoid congestion.

6. If furniture to be placed is a large piece, it should be placed with respect to the wall
→ or room, not with respect to other furniture.
7. Be precise and avoid vague spatial terms. Only one reference object is allowed per
→ placement.
8. Remember 'next to' could only be used when the reference object is a door or window!
9. Wall-mounted items MUST use a wall as their reference object with "hang" as the position.
→ They CANNOT use other furniture as reference or use "on" relationship.

OVERLAP PREVENTION: Assign a unique placement to every furniture item. Identical placements
→ cause items to occupy the same space and overlap. You must ensure all item
→ placements are distinct.

DOOR & WINDOW CLEARANCE: Do NOT place any furniture that overlaps with or blocks a door or
→ window. Doors need their full swing/passage area kept clear. Windows should not be
→ obstructed by furniture placed directly in front of them. The image shows doors
→ (red) and windows (blue) -- use the room dimensions to judge available space and
→ ensure furniture does not encroach on these openings.

BUBBLE INTEGRITY: A bubble is an atomic unit. Do NOT generate separate '<reas>' blocks for
→ individual furniture items contained within a bubble. Only generate one '<reas>'
→ per bubble as a whole. The individual items inside a bubble (listed in its
→ description) are placed automatically relative to the bubble -- you must not place
→ them independently.

</constraint>

WALL-MOUNTED ITEMS

The following items MUST be hung on a wall. For each, select the most appropriate wall based
→ on the room layout, existing furniture, and design principles. These items cannot
→ be placed on the floor or on other furniture.

Wall-mounted items: {wall_furniture_list}

For each wall-mounted item, provide a '<reas>' block explaining your wall choice, then in
→ the extraction step use relationship "hang" with the chosen wall.

<example>
<reas>
Hanging the frame on the pink wall above the sofa creates a focal point and balances the
→ room visually.
</reas>
</example>

<instruction>
For each element in {furniture_list} and in {bubble_list}, do the following:
1. Choose an appropriate placement relative to one valid reference.
2. Justify the placement in a single '<reas>...</reas>' block.

For each wall-mounted item in the wall-mounted items list, do the following:
1. Choose the most appropriate wall to hang it on.
2. Justify the wall choice in a single '<reas>...</reas>' block.

IMPORTANT: Generate exactly one '<reas>' per bubble, one '<reas>' per single furniture item,
→ and one '<reas>' per wall-mounted item. Do NOT generate '<reas>' blocks for
→ individual furniture items inside a bubble.

Repeat for all elements.

OUTPUT FORMAT REQUIREMENT:
- Your output MUST consist ONLY of '<reas>...</reas>' blocks, one per element.

```

- Do NOT output JSON, numbered lists, markdown, or any other format.
- Do NOT wrap your response in “`json or “`” code fences.
- Each ‘<reas>’ block should contain a brief placement explanation referencing the chosen
  ↪ wall/corner/furniture.
</instruction>

```

A.4 Structured Extraction

Parses the semantic suggestions into executable `place_object` calls, enforcing the admissibility rules described in the main paper (“Structured Extraction”).

```

<instruction>
You are generating structured placement functions for both individual furniture items and
  ↪ functional bubbles based on provided suggestions.

REFERENCE OBJECT DEFINITIONS
Walls: [{wall_list}]
Windows: [{window_list}]
Doors: [{door_list}]
Room: "room"

The room contains the following elements to be placed:
- Bubbles: {bubble_list}
- Single Furniture: {furniture_list}
- Wall-Mounted Items (must be hung on a wall): {wall_furniture_list}

Wall information: {walls_dimensions_query}
Window information: {window_information}
Door information: {door_information}

For each placement suggestion, extract the necessary information and format it using the
  ↪ function call shown below.
Do **not** include any reasoning, explanation, or extra text in your output.
</instruction>

<constraint>
Follow these formatting rules and placement constraints:

1. Use the exact format below for each element placement:
  <func> place_object("element name", [length, height, width], ["reference object"],
    ↪ ["relative position"]) </func>
2. Valid reference objects:
  - Previously placed furniture or bubbles (names must exactly match those from the input
    ↪ lists or suggestions)
  - Walls: [{wall_list}]
  - Room: "room"
  - Windows: [{window_list}]
  - Doors: [{door_list}]
3. Valid relative positions based on reference type:
  - If the reference object is furniture or a bubble: ["in front", "left side", "right
    ↪ side", "on", "under"] (Note: This is typically for single furniture, not bubbles).
  - If the reference object is room: [{corner_list}]
  - If the reference object is wall: ["center", "hang"]
  - If the reference object is window: ["next to"]
  - If the reference object is door: ["next to"]
  NOTE: "hang" is ONLY for wall-mounted items. Floor furniture uses "center" for walls.
3a. Disambiguation of "left side" and "right side":
  - Assume a clockwise traversal of the room perimeter.
  - When conceptually reaching a referenced furniture item during this clockwise
    ↪ traversal, the side encountered first is its "right side"; the opposite lateral
    ↪ side is its "left side".
  - Always apply this rule; do not infer viewer-facing orientation.
4. If the element size is not specified, use: [0, 0, 0]
5. Use only one reference object per function call.

```

```

6. Remember 'next to' could only be used when the reference object is a door or window!
7. If the suggestion contains invalid phrasing, correct it:
   - Example: "center of red wall" → "center", "red wall"
8. Do not include any extra commentary or formatting outside the <func>...</func> blocks.
9. Corner positions in {corner_list} are only allowed when the reference object is "room".
# (Removed: The value "center" is also only allowed when the reference object is "room".)
10. Door Placement rules:
    - ANY wall that contains a door CANNOT have an element placed at the "center" of that
      ↪ wall.
    - If a suggestion places an element at the "center" of a wall with a door, you MUST
      ↪ replace the placement to use "next to" the door as the position, and the door as
      ↪ the reference object.
11. Bubble Placement Rules:
    - Bubbles can only be placed relative to the room, a wall, a window, or a door. They
      ↪ CANNOT be placed relative to other furniture or bubbles.
    - If a suggestion places a bubble relative to another furniture item, re-evaluate and
      ↪ place it relative to the nearest valid wall, corner, window, or door.
12. Uniqueness for Bubbles:
    - Only one bubble may occupy a specific wall center.
    - Only one bubble may occupy a specific room corner.
    - If a duplicate occurs, reassign the later bubble to the first available unused valid
      ↪ location (prefer unused wall centers on walls without doors, then unused room
      ↪ corners).
13. Bubble Integrity:
    - A bubble is an atomic unit. Do NOT generate separate '<func>' calls for individual
      ↪ furniture items contained within a bubble.
    - Only generate one '<func>' per bubble as a whole. The individual items inside a bubble
      ↪ are positioned automatically.
14. Wall-Mounted Items:
    - Items listed in the wall-mounted items list MUST use a wall as the reference object and
      ↪ "hang" as the relative position.
    - They cannot be placed on the floor, on other furniture, or in a corner.
</constraint>

<example>
<func> place_object("sleeping area_1", [2500, 1000, 2000], ["east wall"], ["center"]) </func>
<func> place_object("sofa_1", [2200, 800, 950], ["room"], ["southwest corner of"]) </func>
<func> place_object("work_lamp_1", [200, 370, 200], ["desk_1"], ["on"]) </func>
<func> place_object("frame_1", [800, 600, 30], ["pink wall"], ["hang"]) </func>
</example>

<instruction>
Using the suggestions provided in: {suggestions}, do the following for each element (bubble
  ↪ or single furniture):
- Identify the element name.
- Extract or infer the size (use [0, 0, 0] if not available).
- Determine the correct reference object.
- Correctly identify the relative position, applying all constraints and rules.

Format each result as a separate function call using the required syntax. Only return the
  ↪ <func>...</func> blocks, one per item, in the original order. Do NOT generate
  ↪ <func> calls for individual furniture items inside a bubble.

CRITICAL: You MUST produce exactly one <func> block for EACH of the following items (use the
  ↪ EXACT names shown): {required_items_list}
If a suggestion is missing or unclear for an item, infer a reasonable placement from the
  ↪ room context, but you MUST still output a <func> for it. The total number of <func>
  ↪ blocks must equal the number of items listed above.
</instruction>

```

A.5 Wall-Mounting Classification

A preprocessing call that classifies each inventory item as wall-mounted or floor-standing; wall-mounted items are deferred to the end of the placement order.

```

<instruction>
Determine for each furniture item in the list whether it should be wall-mounted (hung on /
    ↳ affixed to a wall at elevation) or placed on the floor.

Return one <func> block per item using ONLY this format:
<func> set_mounting("item name", true|false) </func>

Definitions:
- Wall-mounted (true): Items designed to hang or attach vertically (e.g., frame, artwork,
    ↳ painting, poster, mirror, wall shelf, floating shelf, wall cabinet, wall-mounted
    ↳ TV, wall lamp, sconce, coat rack, pegboard, hook rail).
- Floor (false): Freestanding or resting items (sofa, chair, armchair, bed, table, desk,
    ↳ side table, nightstand, dresser, wardrobe, floor lamp, rug, bench, coffee table,
    ↳ ottoman, storage box).

Rules:
1. If the name clearly indicates a wall-hung variant (e.g., "wall shelf", "floating shelf",
    ↳ "wall cabinet", "wall lamp", "sconce"), classify as true.
2. Generic terms ("shelf", "cabinet", "lamp") are false unless explicitly prefixed by
    ↳ wall/floating/sconce.
3. TV:
    - "wall-mounted tv" or "tv (wall)" → true
    - plain "tv" → false
4. Ambiguous or unknown items default to false.
5. Mirrors/frames/art are always true unless specified as "floor mirror".
6. Coat rack / pegboard / hook rail → true.
7. Do not infer mounting from size if not stated.
8. No additional commentary--only the <func> lines.

Input furniture list: {furniture_list}
</instruction>

<example>
<func> set_mounting("single_furniture_sofa_1", false) </func>
<func> set_mounting("single_furniture_wall shelf_2", true) </func>
<func> set_mounting("single_furniture_frame_large", true) </func>
<func> set_mounting("single_furniture_floor lamp_black", false) </func>
<func> set_mounting("single_furniture_mirror", true) </func>
<func> set_mounting("single_furniture_tv", false) </func>
<func> set_mounting("single_furniture_wall-mounted tv", true) </func>
</example>

<instruction>
Output one <func> block per item in the original order. Only the blocks.
</instruction>

```

B Plausibility Judge Prompt

The full prompt of the VLM judge (Gemini 3.1 Pro) used for the plausibility score, including the complete four-tier grading rubric summarized in the main paper. The judge receives a rendering of the generated layout and returns a score in $\{1, 2, 3, 4\}$ with free-text reasoning in JSON format.

```

The generated scenelayout of the room is shown in the image provided.
Please evaluate the holistic arrangement of all furniture in the room on a scale of 1-4.

**Important:** Evaluate the layout as a whole. A room often contains multiple pieces of the
    ↳ same furniture type (e.g., several chairs) serving different functions. Judge each
    ↳ piece in its own functional context -- a desk chair facing a wall is correct, a
    ↳ dining chair facing a dining table is correct, and an accent chair against a wall
    ↳ can also be correct. Do NOT penalize the layout just because one piece of a type
    ↳ faces a direction that seems wrong for another piece of the same type.

```


****Important -- surplus items:**** The furniture list is ****chosen by the user****, so the room
 ↳ may contain ****more pieces than fit into tidy functional groups**** (extra chairs, an
 ↳ extra table, spare rugs, a stray bench, an extra cabinet, etc.). The generator must
 ↳ place every provided item, so some pieces will inevitably end up lined along walls,
 ↳ in corners, or as leftovers. ****This is an inventory constraint, NOT a design
 ↳ failure.**** Judge the layout primarily on whether the ****main functional groups****
 ↳ (e.g., the dining set, the primary seating area, the bed zone) are coherent. Do NOT
 ↳ lower the score just because a few surplus/extra pieces sit against a wall or don't
 ↳ belong to a clear group -- that is expected and acceptable.

****Important -- open center is fine:**** Furniture pushed toward the walls with an open, empty
 ↳ area in the middle of the room is a ****normal, valid arrangement**** that supports
 ↳ circulation. An empty center is NOT a void, NOT "awkward", and NOT a defect on its
 ↳ own.

****Grading Rubric:****

****1 = Critical Physical or Semantic Failure:****

- ****Floor-standing**** furniture (tables, sofas, beds, chairs) is floating in mid-air with no
 ↳ support, or objects are heavily intersecting with ***each other*** (e.g., a table
 ↳ inside a sofa).
- Furniture extends ****clearly and unambiguously**** beyond the room boundary (out of room).
 ↳ ****The visible floor is the room boundary.**** A piece counts as out of room ****ONLY****
 ↳ when you can clearly see an ****empty gap / void**** (background, blank space, or a
 ↳ different surface that is NOT the floor) directly ****underneath a large portion**** of
 ↳ the piece, so it is unmistakably hovering past where the floor ends. You must
 ↳ actually see floor missing beneath it.
- ****Strongly bias toward IN-BOUNDS. Do NOT flag these very common false positives:****
 - ****Corner / against-wall placement is correct and expected.**** Beds, sofas, and storage
 ↳ are ***supposed*** to sit flush at the perimeter of the floor, with their base aligned
 ↳ to the floor edge. Touching, aligning with, or sitting right at the floor edge is
 ↳ NOT out of room.
 - ****Perspective foreshortening:**** in a 3D interior view the far edge of a large piece
 ↳ (e.g. a bed in a corner) can visually appear to cross the floor outline even when
 ↳ it is fully inside. Do not penalize unless you see an actual empty gap under it.
 - ****Shadows / dark contact patches**** under or beside furniture are NOT part of the object
 ↳ and NOT a void -- ignore them entirely.
 - If the floor is still visible (or hidden by the piece itself) right up to and under the
 ↳ piece, it is inside the room.
 - If you are not near-certain there is empty void beneath a large part of the piece, treat
 ↳ it as inside (when in doubt, IN-BOUNDS). Do not give score 1 for boundary reasons
 ↳ based on a piece merely being ***close*** to the edge.
- ****Note:**** Wall-mounted items (paintings, shelves, wall lamps, mirrors, clocks, TVs) are
 ↳ expected to be elevated on the wall -- this is NOT a physics error.
- ****Score 1 is reserved for PHYSICAL / boundary failures**** (floating, heavy intersection,
 ↳ clearly out of room). Purely semantic problems (poor grouping, broken pairings,
 ↳ scattered furniture) are ****never**** a score 1 -- the worst a semantically poor but
 ↳ physically valid layout can get is a score 2.

****2 = Structural Failure (a MAIN furniture group is genuinely broken):****
 Score 2 only if a ****primary**** part of the layout is genuinely broken -- i.e. ****any ONE**** of
 ↳ the following:

- A ****major piece is not anchored**** -- a bed, sofa, or large storage unit floats in the open
 ↳ middle of the room instead of being placed against a wall where it belongs. (A
 ↳ piece against a wall, even if it looks plain, is fine.)
- ****The single primary functional group does not exist at all**** -- e.g. a living room where
 ↳ the main seating is genuinely not grouped together AND not oriented toward any
 ↳ focal point, or a dining set where the table has essentially no chairs around it. A
 ↳ ***partially*** imperfect group (a couple of chairs angled oddly, one armchair off to
 ↳ the side) is NOT enough -- that is score 3.
- ****Circulation is blocked**** -- you genuinely could not walk from the entry/edge to the main
 ↳ areas without climbing over or squeezing past furniture.

The objects obey basic physics (on the floor, not heavily overlapping, inside the room), but
 ↳ a MAIN group above is genuinely broken.

```

- **Do NOT reach score 2 by accumulating minor issues.** Surplus/extra items along the
  ↳ walls, a spare rug, an odd bench, a few chairs facing slightly off, or an empty
  ↳ center are NOT structural failures -- these stay at score 3.

**3 = Functional and Coherent (DEFAULT -- use this whenever the main groups work):**
This is the default score whenever the **main functional groups are recognizable and broadly
  ↳ sensible**, and none of the score-2 failures apply.
- The primary group(s) read as groups (the dining set is together, the main seating reads as
  ↳ a seating area), major pieces are anchored, and circulation is passable.
- **Minor flaws are fully allowed and expected here** -- surplus/extra items parked against
  ↳ walls, a stray rug or bench, a few chairs angled imperfectly, slight clutter,
  ↳ generic styling, an empty center, or uneven spacing. Several such minor
  ↳ imperfections together still stay at 3 as long as the main groups function.
- It simply lacks a standout, polished, designer-quality touch (otherwise it would be a 4).

**4 = Excellent / Professional Design (polished main groups + a design highlight):**
Score 4 **only if BOTH** conditions hold:
1. **The main functional groups are polished** -- cleanly oriented, well aligned, and good
  ↳ contact, with no awkward orientations or clutter **within the main groups**.
  ↳ **Surplus/extra items neatly parked against walls or in corners do NOT disqualify a
  ↳ **4** -- they are an unavoidable inventory constraint, not a defect, as long as they
  ↳ are tidy and out of the way. AND
2. **At least one clear design highlight**, such as:
  - a balanced / symmetric composition,
  - generous negative space with comfortable, inviting circulation (not merely passable),
  - tightly grouped, naturally aligned furniture relationships,
  - or a clear visual focal point (e.g., a seating group composed around a TV / fireplace
    ↳ as a conversation center).

If the main groups are clean but plain (no highlight), cap it at 3. A score of 4 must be
  ↳ earned by a visible, describable design strength -- keep it reserved for genuinely
  ↳ impressive layouts, not the default.

**Instructions for Response:**
1. **Analyze the Scene:** Briefly describe the room type and identify the main furniture
  ↳ groups and their intended functions.
2. **Check Physics & Boundary:** Are there floating objects or major collisions between
  ↳ objects? For the room boundary, only flag a piece as out of room when you can
  ↳ clearly see an **empty void underneath a large portion of it** (floor visibly
  ↳ missing beneath it). A bed or sofa placed flush in a **corner or against a wall**
  ↳ is correct -- sitting at the floor edge is expected, NOT out of room. Ignore
  ↳ shadows and perspective foreshortening, and when not near-certain, treat the piece
  ↳ as in-bounds. Only a clear, obvious void beneath the piece forces a score of 1.
3. **Check Main Groups:** Identify the **main functional group(s)** (dining set, primary
  ↳ seating area, bed zone) and judge only whether **those** read as coherent groups --
  ↳ roughly together and sensibly oriented (e.g., chairs around the table, coffee table
  ↳ near the sofa, nightstand by the bed). Do NOT penalize **surplus/extra items**
  ↳ parked against walls or in corners, and remember different pieces of the same type
  ↳ may serve different roles. A main group only fails (score 2) if it genuinely does
  ↳ not exist as a group at all -- minor imperfections stay at 3.
4. **Check Connectivity (visual):** Is there a passable open path to move through the room?
  ↳ An **empty/open center with furniture along the walls is fine** and counts as good
  ↳ circulation. Only treat connectivity as a problem (score 2) if furniture genuinely
  ↳ blocks movement across the room.
5. **Final Score:** Output the score in JSON format. A layout that is functional and makes
  ↳ sense overall should score at least 3. **Tie-break rule:** if you are torn between
  ↳ two scores, pick the HIGHER one unless you can name a concrete, specific defect
  ↳ that justifies the lower score.

Your output must be strictly in the following JSON format:
{
  "reasoning": "YOUR DETAILED REASONING HERE",
  "final_answer": 3
}

```